



LOCAL-FIRST AI SECURITY ASSISTANT

HAL Guardian

Software Summary & Demo Review

Code review, threat detection, model comparison and audit logging - designed to run on-device with no cloud inference by default.

Prepared from	HAL Guardian Demo video and transcript
Demo context	Google Gemma Hackathon 2026 proof of concept
Review date	July 21, 2026

Executive Overview

What the demo establishes, who the software serves and why its design matters.

Purpose	HAL Guardian is presented as a proof-of-concept local-first security assistant. It reviews code and untrusted input, compares local AI models, exposes security functions to other agents, and keeps an on-device audit trail.
Core value	The design prioritizes privacy and control: inference, logs, saved prompts, settings and allowlists remain local by default, while potentially risky actions require explicit user involvement.
Intended use	Human analysts can use the Streamlit interface directly, while an external AI agent can invoke the same modules through predictable commands and receive standardized JSON responses.

Demo-confirmed design principles

- Inference stays on the local machine through Ollama; the demo uses a Google Gemma model at a localhost endpoint.
- No API keys, telemetry or remote inference are required by default.
- Model output is displayed for review and is not executed automatically.
- Web fetching is off by default, restricted by an allowlist and requires explicit confirmation before fetched content reaches the model.
- Audit records and application settings persist locally across restarts.

Stakeholder takeaway

HAL Guardian demonstrates a practical pattern for adding AI-assisted security checks without making cloud services the default trust boundary.

Capabilities

Six modules shown in the demo combine analysis, orchestration, accountability and system visibility.

1. Code Guardian

Reviews uploaded source files, pasted code or approved web content. It detects the language, selects a structured prompt, asks the local model for a Markdown review, then extracts a verdict, summary table and individual findings. Findings include severity, reference, description and recommendation. Full results export as JSON or Markdown.

2. Trust Shield

Scans prompts, emails, documents and web pages for manipulative or hidden commands. A deterministic quick scan looks for command language, meta-instructions and encoded payloads such as Base64, hex and URL encoding; payloads can be decoded automatically. An optional local-model deep scan evaluates intent, severity, confidence and recommended action.

3. Model Playground

Provides free-form chat with locally installed Ollama models. The comparator sends the same prompt to two models, displays both responses and can use a selected model as a judge to explain which response performed better. This supports model selection for a specific task.

4. Sub-Agent Console

Exposes each module as a callable command for another AI agent. Responses use a consistent JSON envelope containing agent, command, target, timestamp, duration, data and error fields. The included prompt guide is intended to let an external agent delegate checks with short, predictable instructions.

5. Audit Engine

Stores local action records and errors for accountability and troubleshooting. Structured outputs can support downstream workflows such as CI pipelines, ticket systems or secondary review processes.

6. Health & Settings

Health verifies that the local Ollama service is reachable, persistent memory is active and errors are visible. Settings control optional web fetching and local configuration. A manual explains the modules and how to add additional local models.

How It Works

A straightforward local architecture keeps analysis, persistence and control on the device.

	User or external agent submits code, text, a document, prompt or an approved URL.
	A Streamlit front end routes the request to Python modules that prepare prompts, perform deterministic checks and call Ollama.
	The selected local Gemma model analyzes the material through a localhost endpoint (shown in the demo as port 11434).
	HAL Guardian parses the response into a verdict, structured findings or a standard agent response envelope while preserving the full model output for verification.
	Results are presented for human review, can be exported as JSON or Markdown, and are logged locally. Suggested fixes must be copied into the user's own project; the application does not write changes automatically.

Local persistence

The demo states that audit entries, saved prompts, web-fetch settings and allowlists survive restarts through local SQLite and JSONL storage.

Integration potential

Because Code Guardian and Trust Shield can return structured results, the concept could fit as a pre-flight security check before an AI agent accepts generated code, fetches a URL or trusts a system health report.

Controls, Requirements & Boundaries

The demo intentionally emphasizes controlled operation rather than autonomous enforcement.

Data locality	Prompts, analyzed content, inference and logs remain on the device by default.
Execution safety	Model suggestions are never executed automatically; the user reviews and applies any change.
Web access	Disabled by default. Enabling it presents a security warning, uses an allowlist and requires explicit confirmation before content is analyzed.
Output verification	Parsed summaries are accompanied by the full Markdown response so the original model output can be checked.
Operational visibility	Audit and health views surface activity, errors, local model reachability and persistent-memory status.

Demo requirements

- A machine capable of running Ollama and the selected local Gemma model.
- The HAL Guardian Streamlit application and its Python back-end modules.
- Local storage for SQLite/JSONL persistence and exported review files.
- Additional latency allowance when the optional deep scan or multi-model comparison is used.

Important limitation

The presenter explicitly describes HAL Guardian as a proof of concept, not a production security product. The demo validates an architectural approach and working feature set; it does not establish production hardening, benchmark accuracy, regulatory compliance or replacement of professional security review.

Summary

HAL Guardian packages several useful security-assistance patterns into one local interface: code review, untrusted-input classification, model evaluation, agent-to-agent delegation, auditability and health monitoring. Its strongest presentation value is the combination of local AI capability with deliberate human approval gates and structured outputs that can connect to broader software or agent workflows.

Source note

This document summarizes claims and demonstrations shown in the HAL Guardian Demo video and its on-screen transcript. Product names and technical terms were normalized where the automated transcript contained obvious transcription errors.